EPICODE — CYBERSECURITY

Simulazione di UDP Flood Attack

Esercizio pratico su attacchi DoS — Denial of Service

STUDENTE
Giuseppe

MODULO
Network Security

AMBIENTE
Kali + Metasploitable

DATA
13 Maggio 2026

Documento riservato — uso didattico · Epicode

1 Introduzione agli Attacchi DoS

Un attacco **DoS (Denial of Service)** ha l'obiettivo di rendere un servizio, una rete o una macchina **non disponibile** ai suoi utenti legittimi, sovraccaricandola di richieste o sfruttando vulnerabilità di protocollo fino all'esaurimento delle risorse.

> 💡 **DEFINIZIONE**
> Un attacco DoS non mira a rubare dati, ma a *degradare o interrompere la disponibilità* di un servizio. Quando l'attacco proviene da più fonti coordinate, si parla di **DDOS (DISTRIBUTED DENIAL OF SERVICE)**
> .

Tipo	Vettore	Livello OSI	Impatto principale
UDP Flood	Pacchetti UDP casuali	Trasporto (L4)	Saturazione banda / CPU
SYN Flood	TCP SYN senza ACK	Trasporto (L4)	Esaurimento half-open conn.
HTTP Flood	Richieste HTTP legittime	Applicazione (L7)	Saturazione web server
ICMP Flood	Ping massivo	Rete (L3)	Saturazione banda
DNS Amplification	Query DNS amplificate	Applicazione (L7)	Amplificazione traffico

2 Meccanismo dell'UDP Flood

L'**UDP Flood** sfrutta la natura *connectionless* del protocollo UDP: non richiede handshake, quindi l'attaccante può inviare pacchetti a velocità massima senza attendere risposta. La vittima deve processare ogni pacchetto in arrivo e, non trovando alcun servizio in ascolto sulla porta indicata, risponde con un messaggio ICMP "Destination Unreachable" — consumando ulteriori risorse.

> ⚠️ **IMPATTO SUL TARGET**
> Il bersaglio subisce: (1) consumo di banda in ingresso, (2) overhead CPU per processare i pacchetti, (3) generazione massiva di risposte ICMP — creando un effetto moltiplicatore sull'utilizzo delle risorse di rete.

1. **Preparazione del socket UDP**
   L'attaccante crea un raw socket UDP (AF_INET, SOCK_DGRAM) senza stabilire alcuna connessione.

2 **Generazione del payload casuale**

Viene generato un payload di 1 KB (1024 byte) con byte casuali tramite `random.randbytes()`.

3 **Invio massiccio**

Il loop invia N pacchetti verso IP:porta target — ogni `sock.sendto()` è immediato, senza attesa di risposta.

4 **Saturazione del target**

Il server vittima riceve il flood: la porta UDP non ha servizi in ascolto, ogni pacchetto genera overhead di elaborazione.

3 Analisi del Codice Python

Il programma sviluppato implementa tutte le specifiche richieste dalla traccia: acquisizione dei parametri via input, generazione del payload casuale da 1 KB, invio tramite socket UDP e feedback in tempo reale per ogni pacchetto inviato.

udp_flood.py

```python
import socket
import random

target_ip   = input("Inserisci l'IP target: ")
target_port = int(input("Inserisci la porta UDP target: "))
num_packets = int(input("Quanti pacchetti da 1 KB vuoi inviare? "))

packet = random.randbytes(1024)  # 1 KB = 1024 byte casuali

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

print(f"
[*] Avvio UDP Flood verso {target_ip}:{target_port}")
print(f"[*] Pacchetti da inviare: {num_packets}
")

for i in range(num_packets):
    sock.sendto(packet, (target_ip, target_port))
    print(f"[+] Pacchetto {i+1}/{num_packets} inviato")  # ← CORRETTO

sock.close()
print("
[*] UDP Flood completato.")
```

> 📋 **SCELTE IMPLEMENTATIVE**
> Il payload viene generato una sola volta prima del ciclo con `random.randbytes(1024)` per massimizzare le prestazioni di invio. Il socket UDP (`SOCK_DGRAM`) è connectionless: ogni `sendto()` è indipendente e non richiede handshake, rendendo l'invio il più rapido possibile.

4 Esecuzione e Screenshot

Il test è stato eseguito in un ambiente di laboratorio controllato: macchina attaccante **Kali Linux** e macchina bersaglio **Metasploitable 2**, entrambe sulla rete locale `192.168.0.0/24`.

Parametro	Valore
IP Attaccante (Kali)	`kali.Home / 192.168.0.239`
IP Target (Metasploitable)	`192.168.0.6`
Porta UDP target	29343 (casuale)
Dimensione pacchetto	1024 byte (1 KB)
Numero pacchetti inviati	~137.000+
Porta sorgente	55131

Output del programma attaccante — ogni riga conferma l'invio di un pacchetto con numero progressivo e payload casuale:

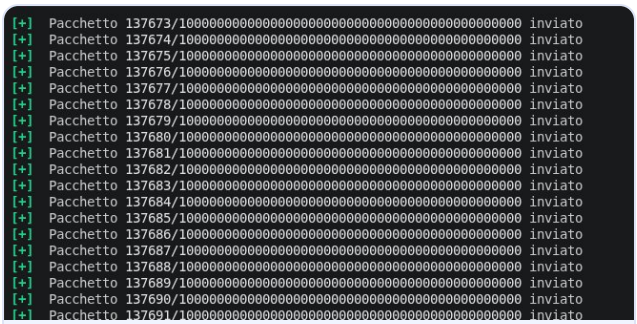

Fig. 1 — Output dell'attacco: pacchetti numerati inviati in sequenza

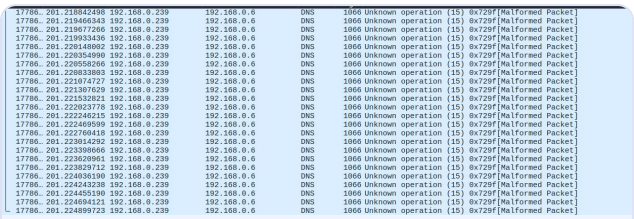

Fig. 4 — Wireshark: pacchetti DNS malformati (opcode 15) intercettati

📊 **NOTA SULLE CATTURE WIRESHARK**

Wireshark classifica i pacchetti UDP come "*DNS — Unknown operation (15) — Malformed Packet*" perché la porta 53 (DNS) è coinvolta nell'elaborazione, ma il payload è byte casuali non conformi al protocollo DNS. Il codice operazione 15 (0x0F) non corrisponde ad alcun opcode DNS standard, confermando la natura casuale del payload.

5 Analisi del Traffico — tcpdump

Sulla macchina Metasploitable è stato eseguito `tcpdump` per catturare il traffico in arrivo durante l'attacco. Le catture confermano la ricezione massiva di pacchetti dalla macchina Kali.

Fig. 2 — tcpdump: flood di pacchetti UDP da kali.Home:55131 verso 192.168.0.6

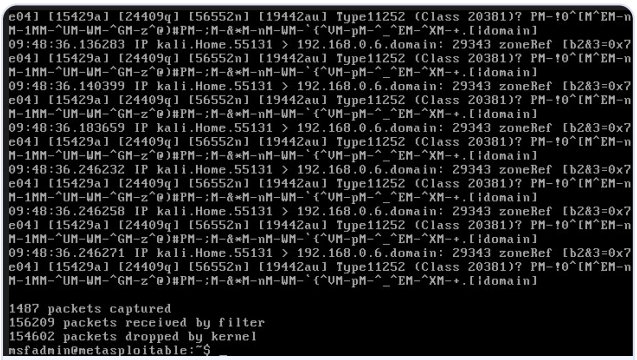

Fig. 3 — tcpdump: statistiche finali — 1487 catturati, 154.602 droppati dal kernel

🚨 **DATO CRITICO — PACKET DROP**

Le statistiche di tcpdump mostrano:
156.209 PACCHETTI RICEVUTI
dal filtro, di cui
154.602 DROPPATI DAL KERNEL
(99%) e solo 1.487 effettivamente catturati. Questo dimostra concretamente l'efficacia del flood nel sovraccaricare lo stack di rete della vittima — il kernel non riesce a processare i pacchetti abbastanza velocemente da consegnarli a tcpdump.

Metrica tcpdump	Valore	Significato
Pacchetti catturati	1.487	Processati da tcpdump
Pacchetti ricevuti dal filtro	156.209	Arrivati allo stack kernel
Pacchetti droppati dal kernel	154.602	Persi per overflow buffer
% drop rate	**99.0%**	Indicatore di saturazione

6 Conclusioni e Contromisure

L'esercizio ha dimostrato con dati concreti come un semplice script Python di poche righe possa generare un carico significativo su una macchina target, con un tasso di packet drop superiore al 99% sul sistema vittima.

> ✅ **OBIETTIVI RAGGIUNTI**
> Tutti i requisiti della traccia sono stati implementati: input dell'IP e porta target, generazione di payload casuale da 1 KB con `random.randbytes()`, configurazione del numero di pacchetti, utilizzo corretto dei socket UDP, e conferma real-time di ogni invio.

Contromisura	Tipo	Efficacia
Rate limiting sul firewall	Network	Alta
Blackholing IP sorgente	Network	Alta
Aumentare buffer kernel (SO_RCVBUF)	Sistema	Media
Intrusion Detection System (IDS)	Monitoraggio	Media
CDN / Scrubbing center	Infrastruttura	Alta

> ⚠️ **AVVERTENZA ETICA E LEGALE**
> Questo esercizio è stato condotto esclusivamente in un ambiente di laboratorio isolato, su macchine virtuali di proprietà. Eseguire attacchi DoS su sistemi reali senza esplicita autorizzazione scritta è un
> **REATO PENALE**
> ai sensi dell'art. 635-bis e 617-bis del Codice Penale italiano e delle normative europee sulla cybercriminalità.